

SUDARSHAN — COMPLETE CODEBASE INTELLIGENCE & EVALUATION GUIDE

0. Executive Summary

SUDARSHAN is a sophisticated, AI-assisted autonomous mobile fraud investigation platform. It automatically decompiles Android applications, dynamically analyzes them in an instrumented Frida sandbox, explores their UI autonomously using an agentic loop, evaluates their visual presentation for impersonation, correlates with threat intelligence, and calculates a deterministic risk score—using a Language Model solely for explanation, not scoring. The system consists of a FastAPI backend orchestrator, an isolated Analysis Engine running Frida/ADB, a React frontend, and a heavy shared python library (`sudarshan_core`) that houses the domain logic.

1. How To Read This Repository

SUDARSHAN's architecture isolates heavy, potentially dangerous analysis tools from the orchestration logic.

- The `backend/` handles HTTP APIs, auth, queueing, the SQLite persistence layer, and RAG/AI explainability.
- The `analysis-engine/` is the heavy lifter. It's a microservice that runs Android-specific tools (Apktool, JADX, ADB, Frida) and runs the sandboxing sessions.
- `shared/sudarshan_core/` is the actual engine. It is mounted into both containers and contains all the risk logic, Frida handlers, and deep UI exploration code.
- `frontend/src/` contains the React UI, heavily leveraging contexts for investigation state.

2. Repository Tree

```
Sudarshan/
├── analysis-engine/      [P0] Analysis microservice (Ubuntu 24.04, JD
│   ├── app/
│   │   ├── adb_bootstrap.py [P2]
│   │   ├── engines/        [P1] Tool bridges (MobSF, Androguard)
│   │   └── main.py         [P0] Microservice orchestrator (FastAPI)
├── backend/            [P0] API Gateway / Orchestrator
│   ├── app/
│   │   ├── ai/            [P1] RAG construction, AI integration
│   │   ├── auth/         [P1] JWT, RBAC
│   │   ├── db/           [P1] SQLite WAL schemas
│   │   ├── routes/       [P1] API definitions
│   │   └── workers/      [P0] analysis_queue.py
│   └── tests/           [P3] Pytest suite
├── shared/sudarshan_core/ [P0] Core Domain Logic (Heart of the system)
│   ├── analyzers/       [P1] Static analysis (apk_analyzer.py)
│   ├── engines/         [P0] Frida sandbox, risk engine, agentic_exp
│   └── models/          [P2] Schemas
```

└─ visual_evidence/	[P1] VIDE pipeline
─ frontend/src/	[P1] React SPA
─ tests/	[P3] Root pytest suite (150 files)
─ scripts/	[P4] CI, bootstrap, validation (validate_cor
─ docs/	[P5] Documentation
─ docker-compose.yml	[P4] Deployment orchestration

3. Repository Architecture

The architecture is fundamentally a master-worker paradigm split across containers to ensure the gateway (backend) never crashes if an analysis tool (analysis-engine) OOMs or hangs while processing malformed malware.

FRONTEND (React) <-> BACKEND (FastAPI, SQLite) <-> ANALYSIS ENGINE (FastAPI, Frida, ADB) | SHARED CORE (Python) | Android Emulator (Genymotion/AVD)

4. Runtime Entry Points

- **Backend Orchestrator:** backend/app/main.py -> Runs uvicorn exposing port 8000.
- **Analysis Worker:** backend/app/workers/analysis_queue.py -> Async event loop processing jobs.
- **Analysis Engine:** analysis-engine/app/main.py -> Runs uvicorn exposing port 8001.
- **Batch Worker:** backend/app/workers/batch_worker.py -> FIFO queue for batch processing.

SUDARSHAN Backend Architecture Analysis

This document provides a detailed architectural analysis of the SUDARSHAN backend (located in backend/app), focusing on execution paths, security features, AI/RAG implementations, and the core responsibilities of its components.

1. End-to-End Execution Path of a File Upload

The file upload and analysis process operates as a sophisticated pipeline, combining synchronous orchestration with asynchronous worker dispatching.

API Routing (backend/app/routes/upload.py)

- **Endpoints:** The upload functionality is primarily exposed via POST /api/v1/analyze (synchronous) and POST /api/v1/analyze/async (asynchronous).
- **Orchestration (_run_analysis_pipeline):**
 - **Delegation:** Initially attempts to offload analysis to the analysis-engine microservice via _call_analysis_engine.
 - **Fallback Execution:** If the microservice is unavailable or times out, it falls back to a local pipeline orchestrating static analyzers (MobSF, Androguard, APKTool,

- JADX) and dynamic analysis (Frida sandbox via `run_frida_analysis`).
- **Enrichment:** Upon receiving static/dynamic findings, the results are run through a risk classification engine and an LLM synthesis step (`analyze_with_llm` in `gemini_client.py`).
- **Persistence:** Results are finalized and saved by calling `_persist_and_index`, which saves the case to the SQLite database (`save_case`), caches the report, and indexes the evidence for RAG (`build_investigation_index`).

Worker Dispatch (`backend/app/workers/analysis_queue.py`)

- **Queueing:** A request to analyze an APK asynchronously invokes `enqueue(job_id, temp_path, filename, sha256_hash, analyst_id)`. This places an item on a `bounded asyncio.Queue`.
- **Worker Execution (`_worker`):**
 - On startup, a configurable number of background worker tasks (`ANALYSIS_WORKERS`) are spun up.
 - These workers pull jobs from the queue and invoke `_run_analysis_pipeline`.
 - State tracking is managed in-memory via an `OrderedDict` named `_jobs` with LRU eviction `_evict_finished_jobs` to prevent memory leaks (OOM).
 - The worker updates job status in SQLite via `upsert_analysis_job`.

Database Layer (`backend/app/db/database.py`)

- Uses `aiosqlite` for asynchronous SQLite interaction, leveraging WAL mode for concurrency (`PRAGMA journal_mode=WAL`).
- `save_case` persists the summarized analysis findings in defined columns (e.g., `final_risk_score`, `family_classification`) along with the complete raw JSON dump in `raw_result`.
- Job states are tracked durably in the `analysis_jobs` table, ensuring restart-safe polling capabilities.

2. Security Features & Implementation (JWT & RBAC)

Authentication and authorization are centrally managed within `backend/app/auth/auth.py`.

- **JWT Authentication:**
 - Leverages `python-jose` for encoding and decoding tokens using HS256.
 - The secret key (`JWT_SECRET_KEY`) is strictly required in production; the system throws a runtime error if it is not provided, avoiding default insecure keys.
 - The login endpoint `POST /api/v1/auth/login` checks credentials against `users` table via `bcrypt` hashing (`passlib`) and issues a Bearer token containing the user's ID, username, and role.
- **Role-Based Access Control (RBAC):**
 - Users have roles: `analyst` (default), `soc_lead`, and `admin`.
 - Defined in FastAPI dependencies via a factory function `require_role(*roles: str)`.
 - Dependencies `require_analyst`, `require_soc_lead`, and `require_admin` are injected into route handlers to restrict access.

- **Privilege Escalation Prevention:** `POST /api/v1/auth/register` explicitly forbids role setting. All self-registered users are assigned the analyst role. The only way to elevate a user is via `PATCH /api/v1/auth/users/{user_id}/role`, which strictly depends on `require_admin`.

3. AI and RAG Implementations

The `backend/app/ai` and `backend/app/rag` directories form the "Sudarshan Investigation Assistant," an evidence-aware Retrieval-Augmented Generation subsystem using Gemini 2.5 Flash.

Architecture & Capabilities

- **Investigation Graph (`gemini_rag.py`):** `build_investigation_index` ingests raw analysis output and chunks it into specific sections (e.g., `permissions`, `network`, `dynamic_findings`, `threat_intelligence`). It maintains an LRU-bounded in-memory index (`_investigation_index`).
- **Knowledge Base (`knowledge_base.py`):** Provides static mappings to MITRE ATT&CK techniques, RBI Master Directions, CERT-In advisories, and malware family profiles based on deterministic flags. This prevents hallucination regarding Indian banking regulatory guidelines.
- **Intent Detection (`detect_intent`):** Classifies the user's question (e.g., "safe", "overlay", "network") and maps it to a prioritized subset of evidence sections, optimizing token usage and context relevance.

Prompt Injection & Hallucination Prevention

- **Sanitization Engine:** The RAG subsystem invokes `sanitize` and `sanitize_block` from `sudarshan_core.engines.agentic.sanitizer` to scrub all APK-derived data (package names, strings, metadata) before they are concatenated into prompt chunks. This mitigates classic prompt injection payloads embedded in malware manifests.
- **Grounding & Segregation:** Gemini NEVER receives the entire raw APK or unformatted report. It only receives context retrieved by `retrieve_evidence`.
- **System Instructions:** The system prompt strongly binds the model:
 - "Answer questions based ONLY on the evidence provided to you"
 - "NEVER invent, guess, or estimate any information not present in the evidence"
 - Instructs the AI that it is *not* the risk scoring engine; it merely explains deterministic outputs.

4. Role of `backend/app/main.py`

`main.py` acts as the FastAPI application entry point and central orchestrator for the backend lifecycle.

- **Initialization & Configuration:**
 - Initializes FastAPI with metadata.
 - Configures strict `CORSMiddleware` (using explicit origins rather than `allow_origins=["*"]` with credentials to conform to standard web security).
 - Tames noisy debug logging from `androguard` to `WARNING` to prevent fatal

memory bloat (OOM kills) when dealing with large DEX parse trees.

- **Router Aggregation:** Registers all API feature routes under `/api/v1/` (upload, batch, discovery, report, cases, intelligence, auth, baselines, etc.).
- **Startup Lifecycle (startup event):**
 - Calls configuration validators (`validate_production_environment`).
 - Calls `init_db()` to ensure SQLite schemas are created and migrated.
 - Seeds the initial admin user. If `ADMIN_PASSWORD` is omitted, it auto-generates a secure, one-time log password.
 - Configures IOC reputation caches, injecting persistence mechanisms into the core engines.
 - Connects the telemetry system (`register_telemetry_sink`).
 - Boots up the asynchronous worker pools for analysis and enterprise batch scanning (`start_workers`, `start_batch_worker`).
- **Shutdown Lifecycle:** Gracefully stops the worker tasks upon termination.

SUDARSHAN Analysis Engine Microservice

- Intelligence Report

1. Executive Summary

The Analysis Engine is a FastAPI REST microservice running in a containerized environment (port 8001), responsible for executing all static and dynamic APK analysis operations. It acts as the workhorse for the SUDARSHAN platform, coordinating tools like Androguard, APKTool, JADX, MobSF, Frida, and ADB. The service is defined in `analysis-engine/app/main.py`.

2. Job Orchestration and Pipeline (main.py)

When an analysis job is received (via `/api/v1/analyze/async` or direct upload `/api/v1/analyze/upload`):

1. **Request Validation:** The APK path is validated against the `UPLOADS_DIR` via `_resolve_upload_path` to prevent path traversal vulnerabilities. File magic checks (`_ZIP_MAGIC` bytes `PK\x03\x04` etc.) are applied during upload streams.
2. **Job Tracking:** An async job is placed in an in-process memory `JOBS` dictionary with status `QUEUED` and a UUID. Finished jobs are periodically evicted based on TTL in `_evict_finished_jobs()`. Because state is in-process, Unicorn/Uvicorn is pinned to 1 worker to prevent 404s on job polls.
3. **Pipeline Execution (`_execute_analysis_pipeline`):**
 - A semaphore `_analysis_slots = asyncio.Semaphore(MAX_CONCURRENT_ANALYSES)` limits concurrency (default 2) to avoid Docker OOM killer events (exit 137) since Androguard parse trees consume massive memory.
 - `PipelineTimer` actively tracks exact phase durations (e.g., `NATIVE_ANALYSIS`, `MOBSF`, `MANIFEST`, `AGENTIC_EXPLORER`).

3. Orchestration of Static & Dynamic Analysis (`main.py`)

- **Parallel Static Analysis:**

- The engine uses `asyncio.gather` to launch Androguard, MobSF, APKTool, and JADX concurrently on separate worker threads.
- Subprocess timeouts (`asyncio.wait_for`) are implemented at each tool level.
- MobSF is treated as optional enrichment; if the `MOBSF_MAX_SECONDS` budget is exceeded, the pipeline raises a timeout internally but catches it to continue without MobSF.

- **Manifest Generation:** Blocking disk I/O generates an investigation manifest storing package and permission details using `build_manifest`.

- **Dynamic Analysis Hand-off:**

- Using `static_findings` (permissions, app labels, flags), the engine bridges static results to the dynamic explorer by passing them to `run_frida_analysis` (in `engines/frida_sandbox.py`). This allows the Frida Sandbox to compare declared permissions against what the app actually requests at runtime.

- **Post-Dynamic Scoring:**

- Threat correlation (`correlate`), mitmproxy HAR ingest (`NetworkCapture.ingest_mitmproxy_har`), and visual/UI evaluation (`safe_run_vide_analysis`) are executed post-sandbox.
- The combined results are sent to `calculate_risk_score` in the risk engine, which returns the `final_risk_score` and risk band (incorporating an AI confidence multiplier).

4. Sandbox Containment Policy (`sandbox_containment.py`)

Container containment ensures that analyzing potentially root-escalated malware on Genymotion/AVD doesn't bridge the guest into the host network. Defined in `sudarshan_core/security/sandbox_containment.py`:

- **Network Isolation:** `frida_listen_host` enforces loopback (`127.0.0.1`) and uses ADB forwarding.
- **Docker Bridge Blocking:** If Genymotion is the provider, setting ADB to `host.docker.internal` is explicitly blocked because it routes container ADB through the Docker host multiplexer, exposing all host services.
- **Subcommand Hardening:** `validate_adb_invocation` blocks commands that widen attack surfaces. For instance, `adb tcpip`, `adb kill-server`, `adb start-server`, and `adb -a` are outright rejected via `ContainmentViolation` exceptions.
- **Fail Closed Strategy:** If `SANDBOX_CONTAINMENT_STRICT=true` or running in production, policy violations fail the analysis rather than just logging warnings.

5. Failure and Retry Logic (`frida_sandbox.py`)

The engine implements resilient design patterns inside `frida_sandbox.py` (`run_frida_analysis` & `_run_device_session`):

- **Sandbox Connection Retries:** Connects via `provider.connect()` up to 3 times with a 2-second sleep if the emulator is initially unresponsive.
- **Concurrency Device Locking:** To prevent races where one job triggers `adb uninstall` while another is hooking, jobs obtain an `asyncio.Lock()` mapped to the emulator's serial (`_DEVICE_LOCKS[device_serial]`).
- **SELinux Management:** If SELinux is "Enforcing" (which blocks Frida's `ptrace` injection), the engine automatically shells out `setenforce 0` to degrade it to Permissive.
- **Installation Fallbacks:** If `adb install` succeeds but `pm path` fails to locate the package, it retries installation once. It gracefully handles "Repaired Derivatives" by dynamically resolving launcher activities (`_resolve_launcher_activity`).
- **Fail-Open Dynamic Status:** If Frida fails to attach or fails to render UI, it uses typed enums (`DynamicAnalysisStatus.INSTRUMENTATION_FAILED` or `NO_UI_RENDERED`). This allows the static report and workflow reconstruction to complete without throwing 500 server errors.

6. Backend Communication

- **Stateless Tool Engines:** The Analysis Engine defines stateless instances (`ApktoolEngine`, `JadxEngine`) so health polls (`/status`) use `is_available` methods via subprocess probes correctly without re-instantiating.
- **Asynchronous Polling Mechanism:** The backend submits a job to `/api/v1/analyze/async` yielding a JSON dict with `job_id`.
- **Status Checks:** The backend then continuously polls `/api/v1/status/{job_id}`, pulling state, `progress_pct`, and eventually the fully serialized dictionary.
- **Authentication:** `_InternalServiceAuthMiddleware` protects all endpoints (except `/health`, `/status`, and `docs`) using the `ANALYSIS_ENGINE_INTERNAL_TOKEN` shared secret header.

Comprehensive Analysis of shared/sudarshan_core

This report provides an in-depth analysis of the `shared/sudarshan_core` directory, which serves as the heart of the Sudarshan system. The analysis focuses on five key subdirectories, traces the full UI exploration loop, and details the Deterministic Risk Engine.

1. Subdirectory Analysis

1.1 analyzers

The `analyzers` directory handles static analysis and relies on tools such as `APKTool`, `JADX`, and `Androguard`. Key findings:

- **`apk_analyzer.py`:** A massive module (21303 bytes) responsible for parsing the APK manifest and structural properties. It likely orchestrates calls to the decompilation engines to extract intents, services, and hardcoded secrets.

1.2 engines

The `engines` directory is the core of dynamic and behavioral analysis.

- **`agentic_explorer.py`** (172KB): The driving force for the Agentic UI exploration loop. It orchestrates UI perceptions and actions using an event-driven or async model (see loop trace below).
- **`ui_explorer.py`** (24KB): The underlying engine for specific UI traversal, likely managing lower-level screen parsing, OCR fallbacks (`_find_ocr_action`), and heuristic deterministic exploration (`_find_deterministic_action`).
- **`frida_sandbox.py`** (210KB): Heavyweight sandboxing that sets up Frida hooks, likely intercepting calls for cryptography, filesystem interaction, and sensitive data access.
- **`behavior_graph.py`**: Maintains a stateful graph of all interactions and observed behaviors, key for constructing the investigation flow.

1.3 models

The `models` directory contains primary data structures.

- **`manifest.py`**: Parses and structures Android manifest details.
- **`schemas.py`**: Defines Pydantic or dataclass schemas for the risk engine, event bus, and AI communication.

1.4 ai

The `ai` directory acts as the prompt interface and language model gateway.

- **`gemini_provider.py`**: Implements the Gemini AI API client.
- **`artifact_explainer.py`**: Suggests components that feed raw technical artifacts (logs, manifests) into the AI to produce human-readable explanations.
- **`gemini_settings.py` & `gemini_errors.py`**: Configuration and error handling for the AI layer.

1.5 visual_evidence

The `visual_evidence` directory manages evidence collection (VIDE) and graph construction.

- **`linker.py`** (31KB): Links visual evidence (screenshots, screen hashes) with behavioral events, reconstructing what the user would see when specific actions trigger.
- **`claim_templates.py` & `quality.py`**: Deals with assessing the quality of evidence or standardizing risk claims.

2. Trace: Full UI Exploration Loop

The UI exploration loop (Perception -> Graph -> Goal -> Action -> Verification) operates primarily within `agentic_explorer.py` and `ui_explorer.py`.

1. Perception:

- `ui_explorer.py` invokes `_dump_ui()` via ADB (`_adb()`) to get the current screen state.
- `_parse_ui()` processes the XML into a list of nodes, and

`_get_screen_hash()` derives a unique identity for the screen.

- `agentic_explorer.py` consumes this via `_screen_observation(obs, classification)`.

2. Graph Construction:

- Observations are attached to a stateful graph (managed by `behavior_graph.py` or within `agentic_explorer.py`'s `_capture_state_frame`). This tracks nodes (screens) and edges (actions).

3. Goal Planning:

- AI and deterministic logic intersect. `ui_explorer.py` has `_find_deterministic_action(nodes)` for known structures and `_find_ai_action(screen_hash, nodes)` (which reaches out via `ai/gemini_provider.py`) to decide the next step if standard heuristics fail. `agentic_explorer.py` formulates complex goals via `_form_recovery_action`.

4. Action Execution:

- `_execute_action(action)` in `ui_explorer.py` runs the chosen tap/swipe. `agentic_explorer.py` has `_execute_with_bounded_retries` for resilient execution.

5. Verification:

- `agentic_explorer.py` performs verification via `_verify_action()` and `_verification_snapshot()`. If an action succeeds or fails, `_resolve_pending_verification()` reconciles the outcome before closing the loop, ensuring the system doesn't enter infinite retry cycles.

3. Trace: Deterministic Risk Engine

The Risk Engine (`engines/risk_engine.py` and `engines/bfci_scorer.py`) applies a strict deterministic model with mathematical boundaries and safety floors.

1. STEI Calculation (Static Threat Exposure Index):

- `_calculate_stei(flags)` in `risk_engine.py` combines axis multipliers (e.g., `_axis_ct`, `_axis_bt`, `_axis_pr`, `_axis_ob`, `_axis_ir`). These functions extract scores based on components, permissions, obfuscation, and intent risk, resulting in a deterministic foundational score.

2. BFCI Calculation (Behavioral Fraud Confidence Index):

- `_calculate_bfci_from_frida(dynamic)` and `bfci_scorer.py` (`score_component`, `detect_fraud_sequences`, `calculate_bfci_v2`) assess dynamic behavior. If Frida hooks observe specific fraud sequences (e.g., reading SMS + connecting to C2), the confidence index spikes.

3. Safety Floors & Exclusions:

- Methods like `_ui_never_rendered(dynamic)` and

`_dynamic_behavior_is_conclusive(dynamic)` define thresholds. If the app crashes instantly or behaves benignly but lacks proper runtime coverage, `dynamic_exclusion_reason` triggers a safety floor, preventing a high-risk app from scoring artificially low due to an execution failure.

4. Execution Assertions:

- `engines/execution_assertions.py` establishes coverage metrics. `build_execution_assertions` calculates assertions that must fire (`all_assertions`, `fired`, `unfired`, `coverage_ratio`). If a required assertion doesn't fire (`incomplete_exercise()`), the engine marks the dynamic phase as flawed, enforcing minimum evidence standards before yielding a final `calculate_risk_score`.

SUDARSHAN Codebase Analysis: Frontend, Tests, and Deployment

This document provides a detailed breakdown of the `frontend/`, `tests/` (`backend/` `tests`), `scripts/`, and root deployment configuration (Docker files).

1. Frontend Architecture (`frontend/src`)

The frontend is a React 18 Single Page Application (SPA) built with Vite. It interacts with the FastAPI backend.

Main Pages and Routing

Routing is handled in `frontend/src/App.tsx` using `react-router-dom`:

- **/ (Upload View):** Maps to `pages/Upload.tsx`. This is the entry point for submitting a new APK for analysis.
- **/fraud-card (Executive Summary):** Maps to `pages/FraudCard.tsx`. Provides the high-level risk breakdown, AI confidence multiplier, and Fraud Risk Score (FRS).
- **/technical (Technical View):** Maps to `pages/TechnicalView.tsx`. Shows detailed dynamic and static findings (e.g., triggered APIs, strings, decoded manifest excerpts).
- **/threat-intel (Threat Intelligence):** Maps to `pages/ThreatIntelView.tsx`. Focuses on C2 servers, IOCs, correlation with external threats, and mitigation advisories.
- **/chat (Investigation Chat):** Maps to `pages/InvestigationChat.tsx`. An interactive chat interface for questioning the AI agent about the current case.
- **/history & /history/:sha256:** Maps to `pages/History.tsx`. Lists previously analyzed cases and provides the ability to reload them into context.
- **/batch & /batch/:batch_id:** Maps to `pages/BatchScan.tsx` and `pages/BatchDetail.tsx` for managing bulk analyses.
- **/login:** Maps to `pages/Login.tsx`. Handles user authentication.

State Management

State is largely managed through React Contexts under `frontend/src/context`:

- **AnalysisContext.tsx:** The core state container for investigations. It stores the active case hash (activeSha256), full case details (analysisResult typed as FraudCardData), dynamic runtime evidence (runtimeEvidenceRaw), and fetches screenshot manifests.
- **AuthContext.tsx:** Manages user authentication status, role, JWT parsing, and handles 401 (unauthorized) API intercepts.
- **InvestigationUIContext.tsx:** Manages UI states specific to the investigation interface.
- **NavDrawerContext.tsx:** Controls the state of the navigation sidebar.

Frontend to Backend API Mapping

The API base URL is resolved via `import.meta.env.VITE_API_URL` (falling back to `http://localhost:8000/api/v1`).

- **/api/v1/auth/login:** Accessed from `Login.tsx` / `AuthContext.tsx`.
 - **/api/v1/cases/:sha256:** Accessed by `AnalysisContext.loadCaseByHash` to retrieve the `FraudCardData`.
 - **/api/v1/cases/:sha256/evidence:** Fetches the dynamic analysis runtime events (`runtimeEvidenceRaw`).
 - **/api/v1/analyze:** Used by `Upload.tsx` for submitting an APK.
-

2. Tests Analysis (backend/tests)

The backend tests are extensive and organized under `backend/tests`.

Covered Categories

1. **AI and Agentic Exploration:** Thoroughly tested the Gemini fallback mechanics, exploration limits, planner caches, and prompt injection (`test_agentic_explorer.py`, `test_boundary_exploration.py`, `test_credential_exploration.py`, `test_deep_exploration*.py`, `test_prompt_injection.py`).
2. **Scoring and Risk Assessment:** Validates the FRS and BFCI logic, static scoring wiring, and classification engine (`test_bfci_scorer.py`, `test_risk_engine.py`, `test_static_scoring_wiring.py`, `test_classification_engine.py`).
3. **Visual Evidence & Forensics:** Extensive test cases for the VIDE subsystem, screenshot linking, and RAG ingestion (`test_visual_evidence*.py`, `test_screenshot_api.py`, `test_screenshot_report_pipeline.py`).
4. **Dynamic Event Pipeline:** Tests how dynamic activities are triggered, parsed, and blocked dynamically (`test_dynamic_event_pipeline.py`, `test_activities_triggered.py`, `test_activity_parser.py`, `test_gateway_dynamic_blocker.py`).
5. **Security Boundaries:** Validates memory bounds, prompt safety, and sandbox execution escapes (`test_boundaries.py`, `test_memory_bounds.py`, `test_hackathon_security_hardening.py`, `test_discovery_security.py`).
6. **Reporting:** Tests generation of PDFs and workflows (`test_pdf_generator.py`, `test_report_generator.py`, `test_workflow_reconstructor.py`).

Missing Coverage

- **analysis-engine/tests:** The engine directly responsible for Frida injection and app orchestration lacks a dedicated `tests/` directory. Testing is primarily handled via backend-driven orchestrator testing.
 - **Frontend / E2E Web Tests:** No Selenium, Cypress, or Playwright UI automation tests exist for the React views.
 - **End-to-End Orchestration Tests in Pytest:** True end-to-end integration tests (spanning MobSF + mitmproxy + dynamic sandbox) are mostly executed as imperative python scripts in `scripts/` (e.g., `test_e2e_pipeline.py`, `validate_real_drinik_e2e.py`) rather than structured pytest suites.
-

3. Scripts Analysis (`scripts/`)

The `scripts/` directory is used heavily for CI/CD, bootstrapping, and E2E validation:

- **Environment Bootstrapping & Setup:** `setup_dynamic_analysis.py`, `health_check.py`, and `bootstrap_sandbox.py` handle ADB integration and ensure Genymotion instances are ready.
 - **E2E Validations:** Full pipeline integration scripts like `validate_real_drinik_e2e.py`, `verify_runtime_pipeline.py`, and `e2e_visual_evidence_validate.py`.
 - **Git Hooks & Maintenance:** PowerShell scripts for repository hygiene (`enable-githooks.ps1`, `strip-cursor-from-git-history.ps1`).
-

4. Deployment Architecture

Standard Development Deployment (`docker-compose.yml`)

The orchestration spins up 5 services connected via a Compose network:

1. **frontend (sudarshan-frontend):** Serves the Vite React app on `5173:5173`. Uses volume bind mounts for HMR (Hot Module Replacement) and enables `CHOKIDAR_USEPOLLING` for Windows/WSL2 compatibility.
2. **backend (sudarshan-backend):** FastAPI Gateway Orchestrator mapping `8000:8000`. Handles API routing, database interactions, auth, and communicates with `analysis-engine` and `mobsf`. Connects to external Gemini, VirusTotal, and OTX via injected `.env` variables.
3. **analysis-engine (sudarshan-analysis-engine):** Heavyweight worker (Ubuntu + Java 17 + Python 3.12). Exposed internally on port `8001`. Responsible for Frida injection (`FRIDA_PORT: 27055`), device interactions via ADB, and AI planning. Uses shared volumes (`uploads`, `mitmproxy_har`, `host ~/ .android`) for zero-copy file processing. Downloads frida server to host cache (`/opt/frida-cache`) to optimize builds.
4. **mitmproxy (sudarshan-mitmproxy):** Transparent proxy sidecar exposing port `8085` natively, dumping HAR files into the `mitmproxy_har` volume used by the engine.
5. **mobsf (sudarshan-mobsf):** The open-source Mobile Security Framework static engine exposed internally on port `8008`.

Production Hardened Deployment (`docker-compose.hardened.yml`)

A secure overlay meant to be executed alongside the main compose file. It dramatically

restricts container privileges:

- **read_only: true:** Locks down container rootfilesystems. Uses tmpfs mounts explicitly for /tmp.
- **cap_drop: ALL:** Drops all Linux capabilities from both backend and analysis-engine.
- **no-new-privileges: true:** Prevents escalating privileges.
- **Seccomp Constraints:** Applies seccomp:deploy/security/seccomp-analysis-engine.json to the analysis-engine to strictly control syscalls.
- **Strict Containment Env:** Enforces SANDBOX_CONTAINMENT_STRICT=true.
- **Removes Developer Mounts:** Prevents live host file mapping, enforcing execution of only baked image contents. Drops ~/.android mount to avoid exposing the host keystore.

27. Data Flow

1. APK Upload -> backend/app/routes/upload.py (SHA256 generated)
2. Job Queued -> backend/app/workers/analysis_queue.py
3. Engine Call -> analysis-engine/app/main.py -> _execute_analysis_pipeline
4. Static Analysis -> sudarshan_core/analyzers/apk_analyzer.py -> Investigation Manifest
5. Dynamic Analysis -> sudarshan_core/engines/frida_sandbox.py -> run_frida_analysis
6. UI Exploration -> sudarshan_core/engines/agentik_explorer.py -> Emits Events
7. Risk Calculation -> sudarshan_core/engines/risk_engine.py -> FRS/STEL/BFCI
8. Persistence -> backend/app/db/database.py -> SQLite
9. Frontend Retrieval -> frontend/src/pages/FraudCard.tsx

28. Dependency Graph

frontend -> backend (API) backend -> analysis-engine (REST / polling) backend -> shared/sudarshan_core (Direct Import - DB models, AI logic, Risk Engine) analysis-engine -> shared/sudarshan_core (Direct Import - Frida logic, Static Analyzers) sudarshan_core.engines.agentik_explorer -> sudarshan_core.engines.frida_sandbox

29. Feature Reality Matrix

FEATURE	IMPLEMENTATION	STATUS
Static Analysis	shared/sudarshan_core/analyzers/	IMPLEMENTED
Dynamic Sandbox	shared/sudarshan_core/engines/frida_sandbox.py	IMPLEMENTED
Agentik Explorer	shared/sudarshan_core/engines/agentik_explorer.py	IMPLEMENTED
Deterministic Risk	shared/sudarshan_core/engines/risk_engine.py	IMPLEMENTED

YARA Scanning N/A (Plumbed but empty rules)

DEAD /
PLANNED

Threat Intel backend/app/routes/intelligence.py

IMPLEMENTED

30. File Importance Matrix

TOP 10 MOST IMPORTANT FILES

1. shared/sudarshan_core/engines/agentive_explorer.py - Core UI exploration loop.
2. shared/sudarshan_core/engines/frida_sandbox.py - Controls instrumentation.
3. shared/sudarshan_core/engines/risk_engine.py - Deterministic scoring math.
4. analysis-engine/app/main.py - Analysis pipeline orchestrator.
5. backend/app/main.py - API Gateway.
6. backend/app/workers/analysis_queue.py - Async job runner.
7. shared/sudarshan_core/analyzers/apk_analyzer.py - Manifest parsing.
8. backend/app/rag/gemini_rag.py - Knowledge indexing for explanations.
9. shared/sudarshan_core/engines/bfci_scorer.py - Dynamic scoring logic.
10. frontend/src/App.tsx - UI Routing.

31. Duplicate / Redundant Code

- **frida-server binary:** 2 identical binaries at frida-server-17.16.4-android-x86_64 and tools/frida-server-17.16.4-android-x86_64. (Cache duplication).
- **test_sample.apk:** Duplicated 3 times (backend/test_sample.apk, test apk/Vulnerable/InsecureBankv2.apk, tests/apks/categories/insecurebankv2.apk).
- **__init__.py:** Widespread empty init files, which is normal for Python but clutters the tree.
- **Preflight logs:** tests/apks/validation_runs/ contains 13 identical preflight.txt files.

32. Potentially Obsolete Code

- scripts/apk_ingest.py
- scripts/live_vide_webview_verify.py

33. Documentation Audit

- **README.md:** AUTHORITATIVE. An excellent summary of the system capabilities.
- **08_DETERMINISTIC_RISK_ENGINE.md:** AUTHORITATIVE. Crucial for understanding scoring math.
- **04_DYNAMIC_ANALYSIS_ENGINE.md:** AUTHORITATIVE. Explains the Agentive Explorer perfectly.

34. Security Audit

- **Sandbox Containment:** Strict (`sudarshan_core/security/sandbox_containment.py`). Blocks `adb tcpip` and routes traffic through loopback.
- **Prompt Injection:** Mitigated via `sudarshan_core.engines.agentic_sanitizer`.
- **JWT Secret:** Hard requirement. No default fallback in production.

35. Failure & Recovery Paths

- **Analysis Engine Crash:** Backend timeout, worker logs failure, frontend sees `FAILED` state.
- **Frida Attach Failure:** Retries, falls back to `SPAWN`, fails open with `INSTRUMENTATION_FAILED` to still allow static reporting.
- **Gemini Unavailable:** Agentic explorer falls back to heuristic exploration (`_find_deterministic_action`).

38. Strongest Engineering Decisions

1. **Deterministic Risk Invariant:** AI writes the narrative, math decides the score. This solves LLM hallucination in malware verdicts.
2. **Fail-Open Sandbox:** If dynamic analysis crashes, the system still produces a static report.
3. **Agentic Loop with Deterministic Fallbacks:** Allows deep exploration of modern apps, but works offline/without API keys via fallback heuristics.

39. Weakest Engineering Decisions

1. **SQLite Database:** Using SQLite with WAL is fine for a prototype, but bounds horizontal scaling.
2. **Missing Frontend E2E Tests:** Extensive Python testing, but zero UI automation.
3. **In-memory Job State:** `analysis-engine/app/main.py` uses an in-memory dictionary for jobs, preventing multiple Uvicorn workers.

40. What Should Be Removed?

- Duplicate APK samples in `tests/apks` and `backend/`.
- Duplicate `frida-server` binary in root.

42. What Must NOT Be Removed?

- `shared/sudarshan_core/engines/execution_assertions.py`. It looks like abstract logic but forms the "safety floors" preventing false negatives.
- `scripts/validate_corpus.py`. Core benchmarking script to prove the math works.

48. 10-Minute Codebase Explanation

SUDARSHAN is a pipeline. A user uploads an APK to the backend. The backend drops a job in an `asyncio.Queue`. A worker picks it up and sends it to the `analysis-engine`. The engine runs Androguard and MobSF for static findings, then boots a Genymotion emulator via ADB. It uses Frida to hook the app, while an `agentic_explorer.py` loop drives the UI

to bypass logins and click buttons. The events are saved to an `evidence_store`. The `risk_engine.py` applies deterministic math to the evidence, emitting a score. Finally, `gemini_client.py` uses RAG over the evidence to write a human explanation, and the React frontend displays it all.

49. 50 + Evaluator Questions & Answers

Q: Why separate backend and analysis-engine? A: To isolate unstable, memory-heavy processes (like Androguard parsing a 100MB APK) from the API Gateway serving users.

Q: Why is risk scoring deterministic? A: Because LLMs are non-deterministic and hallucinate. Security tools require absolute repeatability; identical evidence must yield an identical score.

Q: What happens if malware detects the emulator? A: SUDARSHAN utilizes ART deoptimization during Frida attachment, and the `risk_engine` includes an "Evasion Floor". If the app exhibits zero runtime behavior despite static intents, the system flags it as highly suspicious rather than "Safe".

Q: How do you prevent prompt injection? A: The RAG engine (`gemini_rag.py`) runs all extracted APK strings through a dedicated `sanitizer` that neutralizes injection keywords before giving context to the LLM.

50. Final Repository Assessment

SUDARSHAN is an exceptionally well-architected prototype demonstrating professional-grade separation of concerns. The integration of deterministic risk math with AI-driven UI exploration and RAG reporting represents a novel, highly effective approach to automated malware analysis. While constrained by SQLite and lacking UI tests, the core `sudarshan_core` library is robust, strictly contained, and highly resilient to failure.